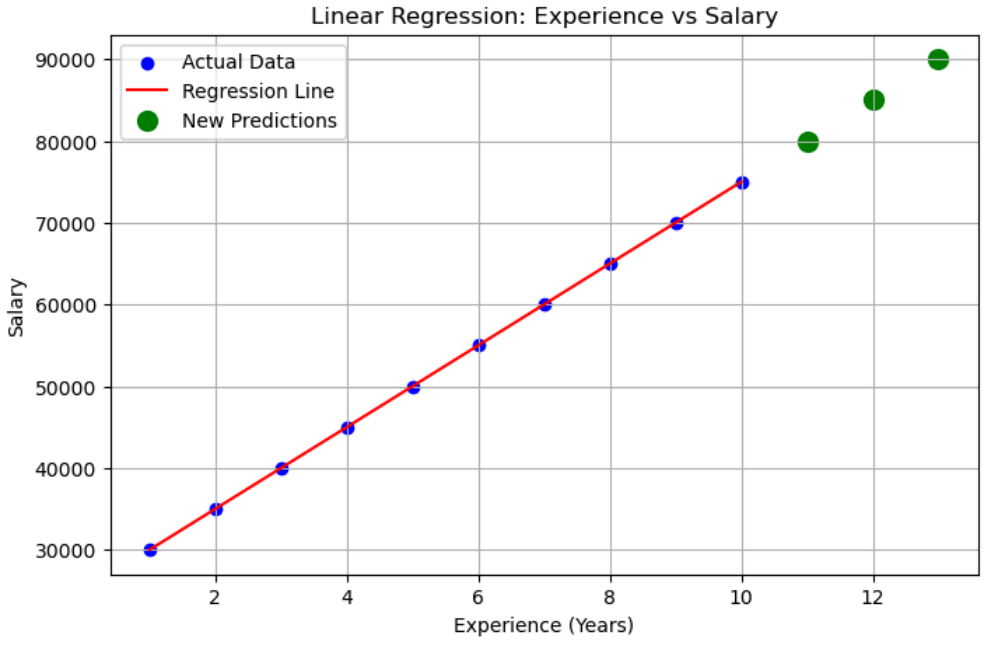
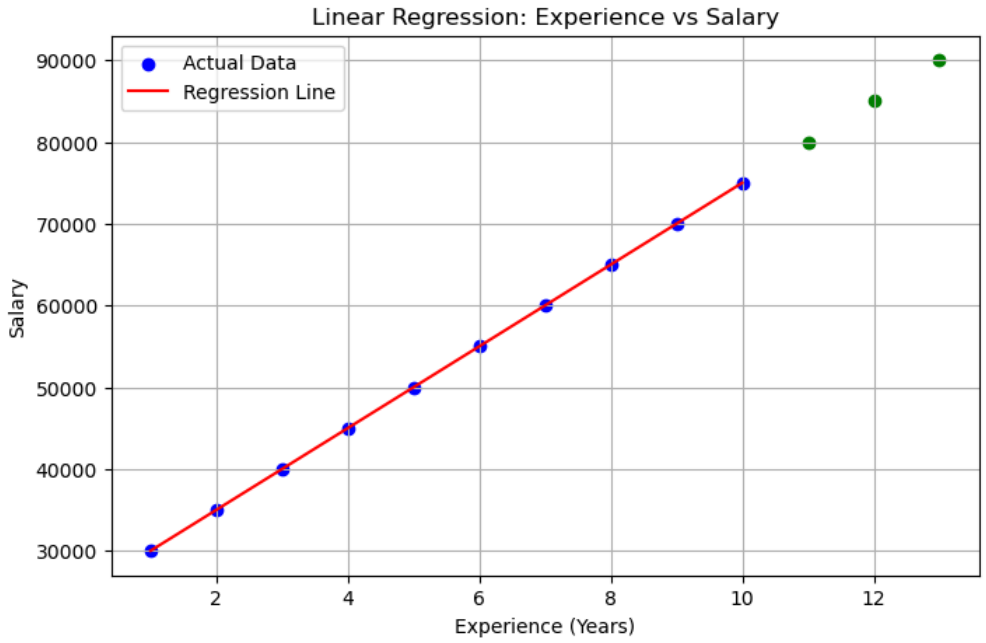
In [3]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
data = {
    'Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Salary': [30000, 35000, 40000, 45000, 50000, 55000, 60000, 65000,
70000, 75000]
}
df = pd.DataFrame(data)
X = df[['Experience']]
y = df['Salary']
model = LinearRegression()
model.fit(X, y)
predictions = model.predict(X)
new_experience = pd.DataFrame({'Experience': [11, 12, 13]})
new_predictions = model.predict(new_experience)
plt.figure(figsize=(8, 5))
plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual
Data')
plt.plot(df['Experience'], predictions, color='red', label='Regression
Line')
plt.scatter(new_experience['Experience'], new_predictions, color='green',
label='New Predictions', s=100)
plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()
```



In [4]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual
Data')
plt.plot(df['Experience'], predictions, color='red', label='Regression
Line')
plt.scatter(new_experience['Experience'], new_predictions, color='green')
plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()
```



In [8]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.DataFrame({
    'Area':[1000,1200,1500,1800,2000],
    'Bedrooms':[2,3,3,4,4],
    'Age':[10,8,5,4,2],
    'Price':[300000,350000,450000,500000,600000]
})
X = df[['Area','Bedrooms','Age']]   #Using multiple features.
y = df['Price']
model = LinearRegression()
model.fit(X,y)
pred = model.predict(X)
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Coefficients: [   250.         -34615.38461538 -13461.53846154]
Intercept: 253846.15384615405
```

In [13]:
```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
df = pd.read_csv('Messy_Employee_dataset.csv')
df_clean = df[['Age', 'Salary']].dropna()
df_clean['Age'] = pd.to_numeric(df_clean['Age'], errors='coerce')
df_clean['Salary'] = pd.to_numeric(df_clean['Salary'], errors='coerce')
df_clean = df_clean.dropna()
X = df_clean[['Age']].values  # Convert to numpy array to avoid warning
y = df_clean['Salary'].values
model = LinearRegression().fit(X, y)
new_ages = np.array([[22], [28], [35], [45], [50]])  # Use numpy array
predictions = model.predict(new_ages)
print(f"Equation: Salary = {model.intercept_:.2f} + {model.coef_[0]:.2f} *
Age")
for age, salary in zip(new_ages, predictions):
    print(f"Age {age[0]}: ${salary:.2f}")
```

```
Equation: Salary = 74552.73 + 322.92 * Age
Age 22: $81656.87
Age 28: $83594.36
Age 35: $85854.76
Age 45: $89083.92
Age 50: $90698.49
```

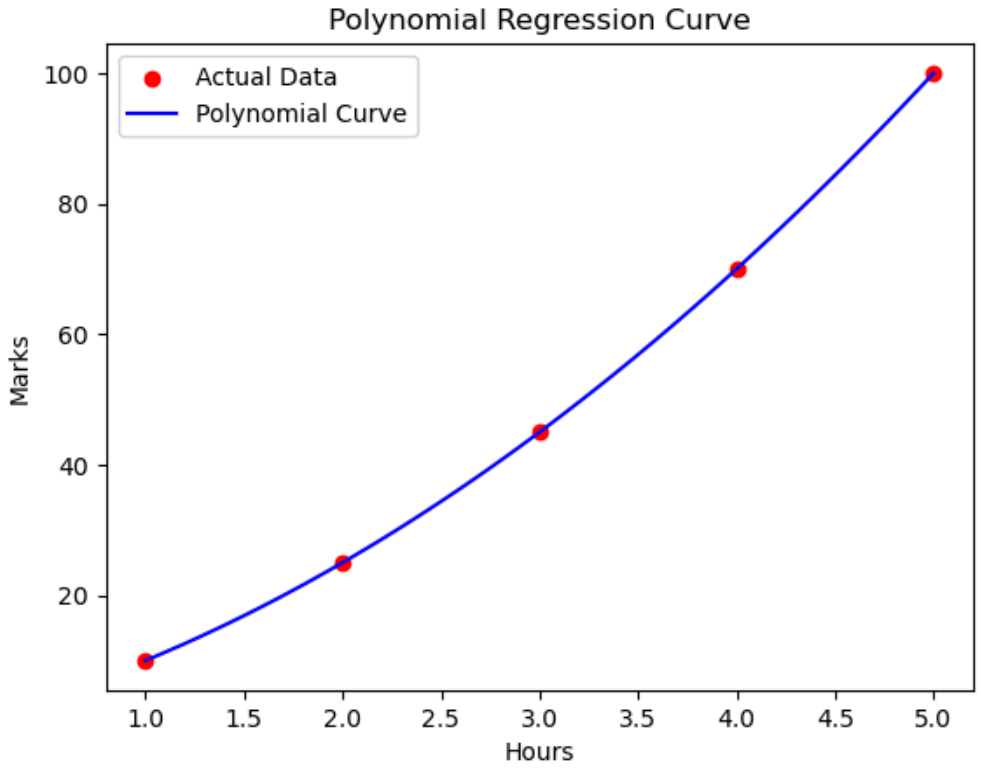
In [17]:
```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor
df = pd.DataFrame({
    'Area': [1000, 1200, 1500, 1800, 2000, 2200, 2500],
    'Bedrooms': [2, 3, 3, 4, 4, 5, 5],
    'Age': [10, 8, 5, 4, 2, 1, 1]
})
X = df[['Area', 'Bedrooms', 'Age']]
vif = pd.DataFrame()
vif['Feature'] = X.columns
vif['VIF'] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]
print(vif)
```

```
    Feature         VIF
0      Area  194.508218
1  Bedrooms  198.832433
2       Age    1.695016
```

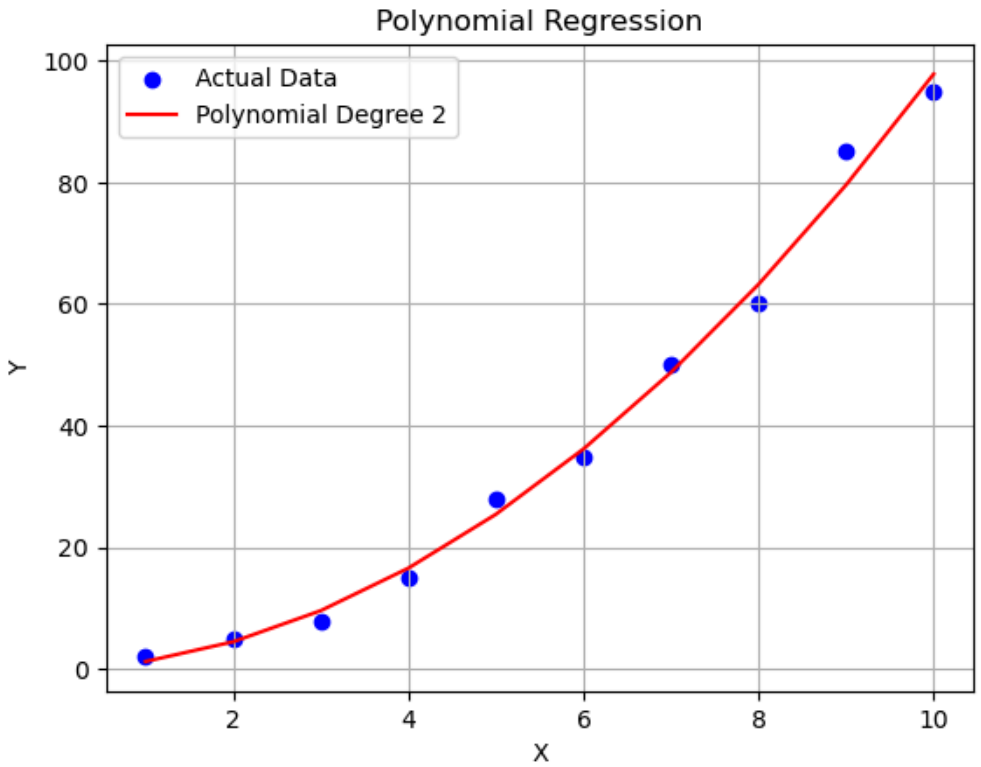
In [20]:
```
import xgboost as xgb
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=5)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, predictions):.4f}")
```

```
Accuracy: 0.9050
```

In [22]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
df = pd.DataFrame({
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [10, 25, 45, 70, 100]
})
X = df[['Hours']]
y = df['Marks']
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)
model = LinearRegression()
model.fit(X_poly, y)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)
y_curve = model.predict(X_range_poly)
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()
```

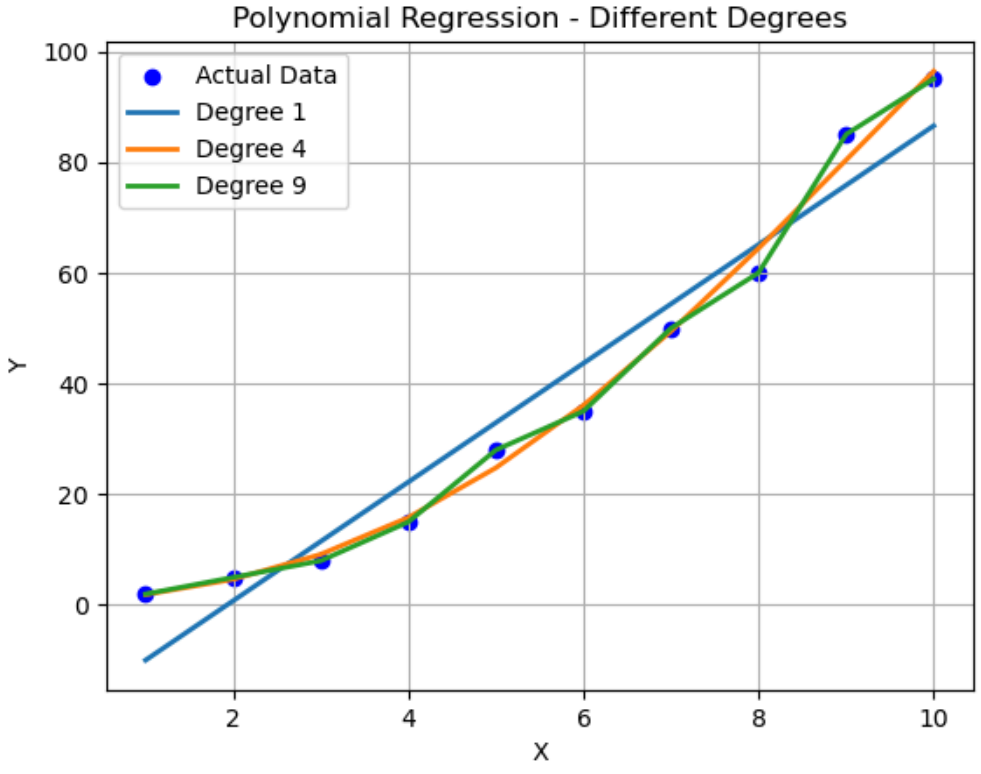


In [24]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([2, 5, 8, 15, 28, 35, 50, 60, 85, 95])
poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(x)
model = LinearRegression()
model.fit(x_poly, y)
y_pred = model.predict(x_poly)
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x, y_pred, color='red', label=f'Polynomial Degree 2')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Polynomial Regression')
plt.legend()
plt.grid(True)
plt.show()
print(f"Intercept: {model.intercept_:.2f}")
print(f"Coefficients: {model.coef_}")
```




```
Intercept: -0.08
Coefficients: [0.         0.42954545 0.93560606]
```

In [27]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([2, 5, 8, 15, 28, 35, 50, 60, 85, 95])
poly1 = PolynomialFeatures(degree=1)
X1 = poly1.fit_transform(X)
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)
poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)
model1 = LinearRegression().fit(X1, y)
model4 = LinearRegression().fit(X4, y)  # Fixed: changed model2 to model4
model9 = LinearRegression().fit(X9, y)
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model1.predict(X1), label="Degree 1", linewidth=2)
plt.plot(X, model4.predict(X4), label="Degree 4", linewidth=2)  # Fixed: X4
instead of X2
plt.plot(X, model9.predict(X9), label="Degree 9", linewidth=2)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Polynomial Regression - Different Degrees')
plt.legend()
plt.grid(True)
plt.show()
print(f"Degree 1 R²: {model1.score(X1, y):.4f}")
print(f"Degree 4 R²: {model4.score(X4, y):.4f}")
print(f"Degree 9 R²: {model9.score(X9, y):.4f}")
```



```
Degree 1 R²: 0.9475
Degree 4 R²: 0.9943
Degree 9 R²: 1.0000
```

Degree 1 → Underfitting - Straight line misses the pattern

Degree 4 → Good Fit - Captures the overall curve 4

Degree 9 → Overfitting -Tries to pass through every point - Learns noise instead of pattern

In [ ]:
